

Advanced Cryptography

Elisabeth Oswald

Secret Sharing

ROADMAP

Conceptual view

Defining secret sharing

Defining security

Some Examples

Splitting the secret

Boolean secret sharing

HISTORY

Secret sharing was a key topic already around the “birth” of modern cryptography in 1979.

Shamir and Blakley both independently developed schemes that enable to share out a secret (the main application was to distribute a key/password/passphrase).

Blakley's scheme can be understood as a variation of Shamir's scheme; it is less efficient.

Our main focus will be on Boolean secret sharing and Shamir secret sharing.

SECRET SHARING: FORMALLY

We want a scheme in which we can “distribute” information about a secret to multiple parties.

Only if “sufficiently” many parties come together, they can reveal the secret.

For instance, five generals hold shares of a secret code to launch as super-duper weapon. Only if at least four generals agree, and come together, can they reveal the secret.

No single general, no pair of generals, and no triplet of generals learns anything about the secret though.

SECRET SHARING: FORMALLY

Secret Sharing Scheme

Given a message space $\mathcal{M}$, a threshold value $t \in \mathbb{N}$, and a number of shares $n \in \mathbb{N}$, with $t < n$.

A (t, n) **threshold secret sharing scheme** consists of two algorithms:

Share: takes a message $m \in \mathcal{M}$ as input, and outputs an n -tuple of randomly selected shares $\mathbf{s} = (s_1, \dots, s_n)$.

Reconstruct: takes a subset of t or more shares as input, and produces a message m' as output.

The example from the previous slide would therefore map to a $(4, 5)$ threshold secret sharing scheme.

Notice the word “randomly” in **Share**: it implies that this is a *probabilistic algorithm*.

SECRET SHARING: FORMALLY

Correctness for secret sharing

A (t, n) threshold secret sharing scheme is **correct** if, for all authorised sets of user $\mathcal{U}$, s.t. $|\mathcal{U}| > t$, and for all $\mathbf{s} \leftarrow \text{Share}(m)$, we have that $\text{Reconstruct}(\{s_i, i \in \mathcal{U}\}) = m$.

In the definition we only require that `Reconstruct` returns something that is from the message space. Obviously, the minimum requirement for any concrete instance, if “enough authorised users” come together, the secret sharing scheme return the secret correctly.

SECRET SHARING: SET UP

Why use/have this weird/complex notation for a “set of authorised users”?

In practice we might want that some parties have more influence than others, so we give them more shares.

For instance, in addition to the five generals, we might also have a president who has some authority in launching the super-duper weapon. Suppose that without his blessing nobody should be allowed to fire the weapon.

How could we do this in a (t, n) threshold secret sharing scheme?

ROADMAP

Conceptual view

Defining secret sharing

Defining security

Some Examples

Splitting the secret

Boolean secret sharing

SECURE SECRET SHARING

Perfectly secure secret sharing

Let Σ be a (t, n) threshold secret sharing scheme with message space $\mathcal{M}$. We say that Σ is **perfectly secure** if for all choices of messages $m_0, m_1 \in \mathcal{M}$, no unauthorised subset of users $\mathcal{U}$ can distinguish between the shares of m_0 and m_1 .

Notice that this definition corresponds to the “indistinguishability” of ciphertext notion (symmetric encryption).

Can you come up with a definition that would capture the “real vs random” world notion of security?

VISUALISING THE DEFINITION

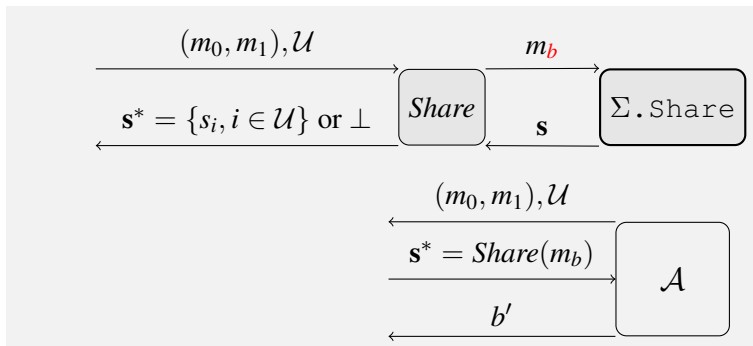


Figure: Exp ind-shares: $\mathcal{A}$ wins if $b' = b$.

$\mathcal{A}$ interacts with a function called *Share* that acts as interface to *Share* of considered secret sharing scheme.

Share returns n shares; *Share* passes i for subset of users $\mathcal{U}$ to $\mathcal{A}$; otherwise returns $\perp$.

ROADMAP

Conceptual view

Defining secret sharing

Defining security

Some Examples

Splitting the secret

Boolean secret sharing

SHARING IS NOT SPLITTING

Some resources use the term *secret splitting* instead of secret sharing.

Take this “literally”, so consider an approach like this:

```
1   $m = m_1 || \dots || m_n$   
2  for i = 1 to n do  
3       $s_i \xleftarrow{\$} m_j$   
4  return  $(s_1 || \dots || s_n)$ ;
```

Listing: An insecure $\Sigma.\text{Share}(m)$

Notice that the definition from before requires that shares are “random”: thus the random assignment from chunks to shares. Obviously this makes reversing the sharing “difficult”.

SPLITTING IS INSECURE

This is obviously insecure: why?

How can we show more formally, i.e. by using the security definition (and associated game) that this scheme is insecure?

ROADMAP

Conceptual view

Defining secret sharing

Defining security

Some Examples

Splitting the secret

Boolean secret sharing

AN (2, 2) BOOLEAN SECRET SHARING SCHEME

We assume that the purpose of the scheme is to work on bit strings. Therefore, we will use the bit-wise addition modulo 2 (aka Boolean addition, aka exclusive-or) to create two shares for an input message.

```
1  def B22.Share(m) :  
2       $s_1 \xrightarrow{\$} \{0, 1\}^l$   
3       $s_2 = s_1 \oplus m$   
4      return (s1, s2)
```

Listing: Boolean (2,2) secret sharing: Share function

```
1  def B22.Reconstruct(s1, s2) :  
2      return (s1  $\oplus$  s2)
```

Listing: Boolean (2,2) secret sharing: Reconstruct function

SET OF AUTHORISED USERS, CORRECTNESS

Sets of unauthorised users are $\{\{1\}, \{2\}\}$ (i.e. no single user should learn anything about s).

The only authorised set is thus $\{1, 2\}$ (it includes both users).

Correctness follows immediately from the construction: the reconstruct function computes $s_1 \oplus s_2 = s_1 \oplus (s_1 \oplus m) = m$.

SECURITY OF (2,2) BOOLEAN SECRET SHARING: INTUITION

s_1 is just a random string that is independently chosen of the message.

s_2 is an exclusive-or between the message and the random string s_1 : essentially it is a one-time pad encryption of m .

We know the one-time is perfectly secret. Ergo s_2 on its own does not reveal any information about m .

We have just covered all unauthorised subsets of parties.

SECURITY OF (2,2) BOOLEAN SECRET SHARING: MORE FORMAL

Case 1: $\mathcal{A}(m, m')$, and $\mathcal{U} = \{1\}$.

Thus $\mathcal{A}$ sees s_1 :

$$\Pr[s_1] = \Pr[s_1|m] = \Pr[s'_1|m'] = 1/2^l,$$

because $\Pr[s_1] = 1/2^l$, and s_1 is independent of m, m' .

Case 2: $\mathcal{A}(m, m')$, and $\mathcal{U} = \{2\}$.

Suppose m was shared: $s_2 = s_1 \oplus m$.

$$\Pr[s_2|m] = \Pr[s_2 = s_1 \oplus m] = \Pr[s_1 = s_2 \oplus m] = 1/2^l.$$

Suppose m' was shared: $s'_2 = s'_1 \oplus m'$.

$$\Pr[s'_2|m'] = \Pr[s'_2 = s'_1 \oplus m'] = \Pr[s'_1 = s'_2 \oplus m'] = 1/2^l.$$

Thus the distribution of s_2 (s'_2) is the same in both worlds.

SECURITY ARGUMENT USING CALLS TO ORACLES

Finally we can frame the same logic in the context of calls to Left/Right oracles and showing that they deliver indistinguishable outputs.

We can do this without reasoning about probabilities explicitly, by arguing how calls to oracles can be replaced without changing the functionality of the oracle.

Below is the code for a “Left oracle” for the Boolean share scheme:

```
1  Share( $m_L, m_R$ ) :  
2      if  $U \geq 2$  return perp  
3       $s_1 \xleftarrow{\$} \{0, 1\}^l$   
4       $s_2 = s_1 \oplus m_L$   
5      return  $\{s_i | i \in \mathcal{U}\}$ 
```

SECURITY ARGUMENT WITH ORACLES, CONT.

We expand the if statement. Notice that even though we are in the left world, we can swap m_L to m_R in the first case because s_2 is not returned.

```
1  Share( $m_L, m_R$ ) :  
2      if  $\mathcal{U} \geq 2$  return perp  
3      if  $\mathcal{U} = \{1\}$   
4           $s_1 \xleftarrow{\$} \{0, 1\}^l$   
5           $s_2 = s_1 \oplus m_R$   
6          return  $s_1$   
7      if  $\mathcal{U} = \{2\}$   
8           $s_1 \xleftarrow{\$} \{0, 1\}^l$   
9           $s_2 = s_1 \oplus m_L$   
10     return  $s_2$ 
```

SECURITY ARGUMENT WITH ORACLES, CONT.

In the second case the code is really the one-time pad encryption of a message m_L . However we proved before that the one-time encryption of a message m is indistinguishable from the one-time encryption of a message m' : so we may swap m_L to m_R .

```
1  Share( $m_L, m_R$ ) :  
2      if  $\mathcal{U} \geq 2$  return perp  
3      if  $\mathcal{U} = \{1\}$   
4           $s_1 \xleftarrow{\$} \{0, 1\}^l$   
5           $s_2 = s_1 \oplus m_R$   
6          return  $s_1$   
7      if  $\mathcal{U} = \{2\}$   
8           $s_1 \xleftarrow{\$} \{0, 1\}^l$   
9           $s_2 = s_1 \oplus m_R$   
10     return  $s_2$ 
```

Thus we end up with a call to the right oracle, yippee!

GENERALISING BOOLEAN SECRET SHARING

It is straightforward to replace “addition modulo 2” with “addition modulo n ”.

It is also straightforward to extend to the (n, n) case.

It is doable to extend this to the $(2, n)$ case by constructing $(2, 2)$ secret sharing schemes for all pairs of users.

Going to a (t, n) scheme is not nice.